

relatorio_AES_SPI

Relatório técnico sobre AES e SPI

Fundamentos da criptografia simétrica e da comunicação serial entre circuitos

O AES e o SPI atendem a necessidades diferentes dos sistemas digitais. O Advanced Encryption Standard transforma dados por meio de uma chave secreta, permitindo sua proteção criptográfica. A Serial Peripheral Interface estabelece a troca de bits entre um controlador e seus periféricos. Em um sistema embarcado, ambos podem coexistir: o AES participa da proteção da informação e o SPI realiza a comunicação entre circuitos.

Este relatório apresenta primeiro a estrutura e o funcionamento do AES, com suas operações internas, expansão de chaves, decifragem e um vetor de teste. Em seguida, descreve o SPI de quatro sinais, a temporização, as configurações com vários periféricos e o estudo de caso de redução de pinos. A discussão final relaciona os dois conteúdos e destaca os cuidados necessários à implementação.

AES e a norma FIPS 197

O AES é uma cifra simétrica de blocos baseada no algoritmo Rijndael. Simétrica significa que a mesma chave secreta fundamenta a cifragem e a decifragem; de blocos significa que cada transformação recebe e devolve uma quantidade fixa de dados. No AES, esse bloco tem sempre 128 bits, equivalentes a 16 bytes. Os nomes AES-128, AES-192 e AES-256 identificam o tamanho da chave, e não o tamanho do bloco.

Variante	Chave	Bloco	Rodadas
AES-128	128 bits / 16 bytes	128 bits	10
AES-192	192 bits / 24 bytes	128 bits	12
AES-256	256 bits / 32 bytes	128 bits	14

A chave seleciona uma transformação reversível entre blocos. Quem possui a chave correta consegue recuperar o texto claro a partir do texto cifrado. O algoritmo pode ser público: o segredo que precisa ser protegido é a chave. Uma chave maior amplia o espaço de busca exaustiva, mas também altera a expansão de chaves e o número de rodadas.

Contexto da edição. A FIPS 197 foi publicada em 26 de novembro de 2001. O aviso anexado ao arquivo informa sua substituição pela FIPS 197-upd1 em 9 de maio de 2023 e declara que a atualização é editorial, sem mudanças técnicas no algoritmo. O arquivamento dessa edição não significa que o AES tenha sido abandonado.

2 Organização dos dados e base matemática do AES

2.1 A matriz de estado

O processamento interno ocorre em uma matriz chamada State, ou estado. Ela possui quatro linhas e quatro colunas, totalizando 16 bytes. Os bytes da entrada são inseridos por colunas: os quatro primeiros ocupam a primeira coluna; os quatro seguintes, a segunda; e assim sucessivamente. Essa convenção é essencial para que uma implementação produza os resultados previstos na norma.

Para a entrada hexadecimal 00112233445566778899aabbccddeeff, o estado inicial fica organizado da seguinte forma. Cada par hexadecimal representa um byte, e a leitura de volta para a sequência também ocorre por colunas.

Linha	Coluna 0	Coluna 1	Coluna 2	Coluna 3
0	00	44	88	CC
1	11	55	99	dd
2	22	66	aa	ee
3	33	77	bb	ff

A norma usa Nb para o número de palavras de 32 bits do bloco, Nk para o número de palavras da chave e Nr para o número de rodadas. No AES, Nb vale sempre 4; Nk vale 4, 6 ou 8; e Nr vale 10, 12 ou 14, respectivamente. Cada coluna do estado corresponde a uma palavra de 32 bits.

2.2 Operações em um corpo finito

Os bytes são tratados como elementos do corpo finito $GF(2^8)$, que contém 256 elementos. Cada bit é um coeficiente de um polinômio binário de grau máximo 7. A soma desses elementos é o XOR bit a bit, sem transporte: bits iguais resultam em 0 e bits diferentes, em 1. Por exemplo, em hexadecimal, 57 XOR 83 = d4.

A multiplicação é uma multiplicação de polinômios com redução pelo polinômio irreducível $x^8 + x^4 + x^3 + x + 1$, representado por 0x1b. Ela não é a multiplicação inteira comum. Essa redução mantém o resultado representável em um byte e permite definir inversos multiplicativos para todos os elementos não nulos.

Uma operação frequente é xtime, a multiplicação por 0x02. Desloca-se o byte um bit à esquerda, retêm-se os oito bits menos significativos e, se o bit mais significativo original era 1, aplica-se XOR com 0x1b. Assim, $xtime(0 \times 57) = 0 \times ae$ e $xtime(0 \times ae) = 0 \times 47$. Multiplicar por 0×03 equivale a aplicar xtime ao byte e combinar o resultado com o byte original por XOR.

Essas operações dão suporte à substituição de bytes e à mistura de colunas. O objetivo de apresentá-las é tornar claro por que um circuito AES usa substituições, deslocamentos e portas XOR em vez de apenas somas e multiplicações convencionais.

3 Transformações e sequência de cifragem

3.1 As quatro transformações

SubBytes. Cada byte é substituído por outro por meio de uma tabela invertível chamada S-box. Sua construção combina a inversão multiplicativa em $GF(2^8)$, considerando zero como zero

nessa etapa, com uma transformação afim que utiliza a constante 0x63. A S-box introduz não linearidade; por exemplo, substitui 0x53 por 0xed e 0x00 por 0x63.

ShiftRows. As linhas do estado sofrem rotações cíclicas para a esquerda. A linha 0 permanece igual; as linhas 1, 2 e 3 são deslocadas em 1, 2 e 3 bytes, respectivamente. Nenhum byte é descartado. Essa redistribuição faz com que bytes de uma coluna passem a interagir com bytes de outras colunas nas operações seguintes.

MixColumns. Cada coluna é transformada em outra coluna por uma combinação linear em GF (2^8). Cada byte resultante depende dos quatro bytes originais daquela coluna. A matriz de coeficientes abaixo mostra essa mistura; os valores são hexadecimais. Todas as saídas devem ser calculadas a partir dos valores originais da coluna.

Saída	Entrada a	Entrada b	Entrada c	Entrada d
Primeiro byte	02	03	01	01
Segundo byte	01	02	03	01
Terceiro byte	01	01	02	03
Quarto byte	03	01	01	02

Por exemplo, o primeiro byte de saída é o XOR de quatro termos: 0x02 multiplicado por a, 0x03 multiplicado por b, c e d. A combinação de ShiftRows e MixColumns promove a difusão das alterações pelo estado ao longo das rodadas.

AddRoundKey. Os 16 bytes do estado são combinados por XOR com os 16 bytes da chave da rodada. Essa é a etapa da cifragem que incorpora diretamente o material secreto ao estado. Aplicar novamente o mesmo XOR desfaz a operação, pois $(a \oplus k) \oplus k = a$.

3.2 Ordem das operações

A cifragem começa com AddRoundKey usando a chave inicial. Em seguida, executa Nr - 1 rodadas com a sequência SubBytes, ShiftRows, MixColumns e AddRoundKey. A última rodada executa SubBytes, ShiftRows e AddRoundKey, sem MixColumns. O estado final é então convertido para a sequência de 16 bytes de saída.

No AES-128, portanto, existem uma adição inicial de chave, nove rodadas completas e uma rodada final. As dez rodadas não incluem a adição inicial. Alterar a ordem das etapas ou incluir MixColumns na última rodada produz um resultado incompatível com o padrão.

4 Expansão de chaves e decifragem

4.1 Geração das chaves de rodada

A expansão de chaves transforma a chave original em uma sequência de palavras de 32 bits, denominada key schedule. Cada aplicação de AddRoundKey utiliza quatro dessas palavras, equivalentes a 128 bits. São necessárias Nr + 1 chaves de rodada, porque a cifra também aplica uma chave antes da primeira rodada.

Variante	Palavras geradas	Chaves de rodada	Total expandido
AES-128	44	11	176 bytes
AES-192	52	13	208 bytes

Variante	Palavras geradas	Chaves de rodada	Total expandido
AES-256	60	15	240 bytes

As primeiras N_k palavras são copiadas da chave original. Para obter cada palavra seguinte, o algoritmo combina por XOR a palavra N_k posições antes com uma transformação da palavra anterior. Na maior parte das posições, essa transformação é apenas a cópia da palavra anterior. Em posições específicas, entram três recursos:

RotWord gira uma palavra de quatro bytes uma posição à esquerda: $[a, b, c, d]$ torna-se $[b, c, d, a]$. SubWord aplica a S-box a cada um desses quatro bytes. Rcon fornece uma constante de rodada, cujo primeiro byte é uma potência de $0x02$ no corpo finito e cujos três bytes restantes são zero.

Quando o índice i é múltiplo de N_k , a palavra anterior passa por RotWord e SubWord e recebe XOR com $Rcon[i/N_k]$, antes do XOR com a palavra N_k posições atrás. No AES-256 há uma regra adicional: quando o resto de i dividido por 8 é 4, aplica-se somente SubWord à palavra anterior antes dessa combinação. Omitir essa condição invalida a expansão de 256 bits.

Expandir a chave não cria entropia adicional: todas as chaves de rodada são determinadas pela chave original. Armazenar a sequência completa economiza recomputação, enquanto gerá-la durante o processamento pode reduzir o uso de memória. A escolha depende da arquitetura e dos requisitos de desempenho.

4.2 Recuperação do texto claro

A decifragem aplica transformações inversas e utiliza as chaves de rodada em ordem inversa. Primeiro, combina o texto cifrado com a última chave. Para as rodadas intermediárias, executa InvShiftRows, InvSubBytes, AddRoundKey e InvMixColumns. Ao final, executa InvShiftRows, InvSubBytes e AddRoundKey com a chave inicial, recuperando o bloco original.

InvShiftRows gira as linhas para a direita; InvSubBytes usa a S-box inversa; InvMixColumns usa uma matriz inversa com os coeficientes $0x0e$, $0x0b$, $0x0d$ e $0x09$ em disposições apropriadas. AddRoundKey permanece igual porque XOR é sua própria inversa. A norma também apresenta uma cifra inversa equivalente, que reorganiza etapas e transforma as chaves intermediárias. Essa alternativa exige respeitar as alterações especificadas.

5 Exemplo do AES e cuidados de implementação

5.1 Vetor de teste AES 128

O apêndice C. 1 da FIPS 197 fornece um exemplo conhecido para verificar uma implementação. Os valores abaixo representam bytes em hexadecimal, e não o texto dos caracteres. O bloco possui 16 bytes e a chave também possui 16 bytes.

```

Texto claro: 00112233445566778899aabbccddeeff
Chave: 000102030405060708090a0b0c0d0e0f
Texto cifrado: 69c4e0d86a7b0430d8cdb78070b4c55a

```

Ponto do processamento	Estado serializado por colunas
Após AddRoundKey inicial	00102030405060708090a0b0c0d0e0f0

Ponto do processamento	Estado serializado por colunas
Após SubBytes da rodada 1	63cab7040953d051cd60e0e7ba70e18c
Após ShiftRows da rodada 1	6353e08c0960e104cd70b751bacad0e7
Após MixColumns da rodada 1	5f72641557f5bc92f7be3b291db9f91a
Após AddRoundKey da rodada 1	89 d 810 e 8855 ace 682 d 1843 d 8 cb 128 fe 4

A chave da primeira rodada é d6aa74fdd2af72fadaa678f1d6ab76fe. Combiná-la por XOR com o resultado de MixColumns gera a última linha da tabela. Comparar esses estados intermediários ajuda a localizar erros de ordenação dos bytes, substituição, aritmética de corpo finito ou expansão de chave.

5.2 O que um resultado correto demonstra

Obter o texto cifrado esperado e recuperar o texto claro pela operação inversa são verificações funcionais importantes. Porém, um único vetor não demonstra a segurança de todo o produto. A norma inclui exemplos para as três variantes, e uma implementação deve verificar todas as variantes que declara suportar. A conformidade formal também não decorre de um teste local isolado.

O AES transforma blocos; a proteção de mensagens de vários blocos depende de um modo de operação e de regras adicionais. A própria apresentação da FIPS 197 prevê seu uso com um modo apropriado. Não basta dividir um arquivo em blocos e cifrar cada um independentemente: blocos iguais, sob a mesma chave, produzem saídas iguais nessa utilização direta, revelando padrões.

Como orientação de projeto, a solução deve tratar também integridade e autenticidade, além de confidencialidade. Esses serviços não são fornecidos pela cifra de bloco isolada. O modo escolhido define requisitos como uso de nonce ou vetor de inicialização, tratamento do comprimento da mensagem e, quando aplicável, geração e verificação de uma etiqueta de autenticação.

A chave deve ser gerada, armazenada e descartada de forma adequada. Tempos de execução, acessos à memória, consumo de energia ou emissões eletromagnéticas podem revelar informação em implementações vulneráveis. Esses cuidados de engenharia complementam a correção matemática do AES; não são resolvidos apenas pelo aumento do tamanho da chave.

6 SPI e transferência serial de dados

6.1 Finalidade e sinais

A Serial Peripheral Interface é uma interface síncrona usada na comunicação entre microcontroladores e circuitos periféricos, como sensores, conversores analógico-digitais, conversores digital-analógicos, memórias e registradores de deslocamento. Síncrona significa que a transferência é coordenada por um sinal de clock. O artigo de Piyu Dhaker concentra-se na configuração de quatro sinais.

O dispositivo que gera o clock é chamado aqui de controlador, correspondente a main no artigo. O dispositivo selecionado é o periférico, correspondente a subnode. O arranjo

apresentado possui um controlador e um ou vários periféricos. Os nomes tradicionais MOSI e MISO indicam o sentido dos dados.

Sinal	Direção	Função
SCLK	Controlador para periférico	Clock que sincroniza a transmissão e a amostragem.
CS ou nCS	Controlador para periférico	Seleciona o dispositivo; ativo em nível baixo nos exemplos do artigo.
MOSI	Controlador para periférico	Transporta dados enviados pelo controlador à entrada SDI do periférico.
MISO	Periférico para controlador	Transporta dados da saída SDO do periférico ao controlador.

Esses quatro sinais não incluem alimentação nem referência de terra, que também precisam ser compatíveis entre os circuitos. As siglas SDI e SDO significam entrada e saída serial do dispositivo em questão; sua interpretação depende do componente observado.

6.2 Como ocorre uma transação

O controlador configura a frequência, a polaridade e a fase do clock de acordo com o periférico. Em seguida, ativa CS, normalmente levando-o a zero. Os bits são apresentados nas linhas de dados e amostrados em bordas específicas de SCLK. Ao terminar a quantidade de bits exigida, o controlador encerra a seleção, respeitando os tempos definidos pelo componente.

O SPI de quatro sinais permite full duplex: enquanto um bit sai em MOSI, outro pode entrar em MISO no mesmo ciclo. Isso não implica que toda informação recebida seja útil. Durante o envio de um comando, por exemplo, a resposta pode ainda não estar disponível. Para continuar uma leitura, o controlador costuma transmitir bits de preenchimento e gerar os clocks necessários.

O tamanho de cada palavra, a ordem dos bits, os códigos de comando, os endereços e os intervalos de seleção dependem do dispositivo. Oito bits por palavra e transmissão do bit mais significativo primeiro são escolhas frequentes, mas não universais. A ficha técnica é a referência para esses parâmetros; conectar os sinais corretos é apenas parte da configuração.

7 Modos SPI e interpretação da temporização

7.1 Polaridade e fase do clock

CPOL define o nível de repouso do clock: 0 significa repouso em nível baixo e 1, em nível alto. CPHA define em qual borda de cada ciclo ocorre a amostragem: com 0, na primeira borda; com 1, na segunda. A outra borda é usada para lançar ou deslocar o próximo bit. Controlador e periférico precisam concordar com a configuração.

Modo	CPOL	CPHA	Repouso	Amostragem	Deslocamento
0	0	0	Baixo	Subida	Descida
1	0	1	Baixo	Descida	Subida

Modo	CPOL	CPHA	Repouso	Amostragem	Deslocamento
2	1	0	Alto	Descida	Subida
3	1	1	Alto	Subida	Descida

Com $CPHA = 0$, o primeiro bit já precisa estar estável antes da primeira borda de clock. Com $CPHA = 1$, a primeira borda lança o bit e a segunda o amostra. Os modos 0 e 3 amostram na subida, mas têm repouso e fase diferentes; o mesmo cuidado vale para os modos 1 e 2, que amostram na descida.

7.2 Exemplo de transmissão no modo 0

No envio de 0xA5, ou 10100101, com o bit mais significativo primeiro, CS permanece baixo e SCLK começa em zero. Os bits são amostrados nas oito subidas. As mudanças para os bits seguintes ocorrem nas descidas, permitindo sua estabilização.

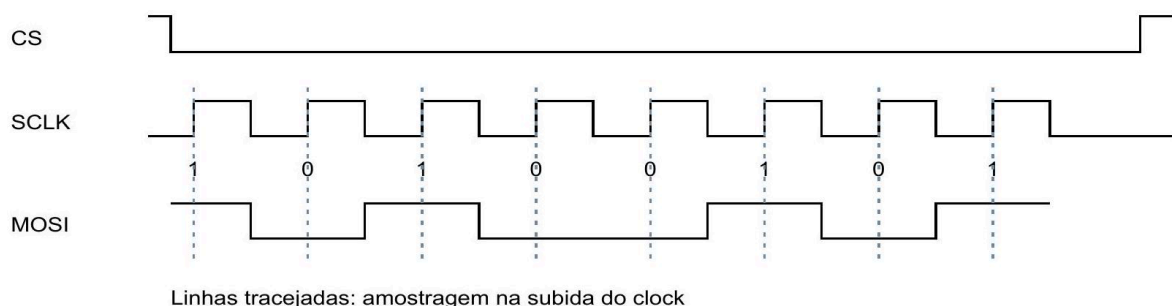


Figura 1. Envio de 0xA5 em MOSI no modo 0. Forma de onda idealizada, sem escala temporal. MISO pode transferir outro byte simultaneamente.

Na figura 2 do artigo, o controlador envia 0xA5 e recebe 0xBA nos mesmos oito ciclos. MOSI e MISO transferem bytes independentes simultaneamente.

7.3 Tempos que precisam ser respeitados

O dado deve permanecer estável antes da amostragem, pelo tempo de setup, e depois dela, pelo tempo de hold. A ficha técnica também limita a frequência, a duração dos níveis do clock, os atrasos de saída e os intervalos entre CS e SCLK. Violar esses tempos pode corromper bits mesmo com CPOL e CPHA corretos.

8 Vários periféricos e redução de pinos

8.1 Seleções independentes

Na configuração regular, SCLK, MOSI e MISO são compartilhados, e cada periférico possui uma linha CS própria. O controlador seleciona o componente desejado e os demais devem liberar a linha MISO, normalmente colocando sua saída em alta impedância. Selecionar ao mesmo tempo dispositivos que dirigem valores diferentes em MISO pode causar contenção elétrica e corromper os dados.

Para N periféricos, a ligação direta típica utiliza três sinais compartilhados e N seleções, totalizando $3 + N$ pinos do controlador, sem contar alimentação ou sinais adicionais. O acesso a um dispositivo é direto, mas a quantidade de pinos de seleção cresce. Decodificadores ou

multiplexadores podem ajudar a gerar seleções, acrescentando componentes e requisitos de controle.

8.2 Conexão em cadeia

No arranjo daisy chain, todos os periféricos compartilham clock e seleção. MOSI alimenta a entrada do primeiro; a saída deste alimenta o segundo, e assim sucessivamente. A saída do último retorna ao MISO do controlador, quando necessária. A transferência percorre os registradores da cadeia, economizando linhas CS.

No exemplo didático do artigo, três estágios de oito bits requerem 24 pulsos para que o byte percorra a cadeia até o terceiro dispositivo. Para carregar três palavras distintas nesse modelo, envia-se primeiro a palavra destinada ao dispositivo mais distante. O número real de bits e o momento de atualização das saídas dependem do protocolo do componente. Nem todo periférico SPI suporta esse arranjo.

Critério	Seleção independente	Daisy chain
Pinos típicos	3 + N	4, no arranjo descrito
Acesso	Direto ao periférico selecionado	Dados percorrem os estágios
Custo temporal	Quadro do dispositivo acessado	Cresce com os bits da cadeia
Compatibilidade	CS individual e MISO compartilhável	Exige suporte explícito à cadeia

8.3 Estudo de caso do artigo

O ADG1412 contém quatro chaves analógicas SPST, cada uma com controle digital próprio. Quatro desses circuitos, formando a matriz 4×4 do exemplo, exigem 16 GPIOs de controle. Um conversor serial-paralelo pode reduzir o uso de pinos, mas acrescenta um componente ao projeto.

A alternativa apresentada é o ADGS1412, com controle por SPI. Quatro dispositivos usam sete pinos no arranjo regular: três compartilhados e quatro seleções. Na configuração em cadeia descrita, usam quatro. O ganho é a redução de conexões do controlador; a atualização passa a depender de uma transferência serial. O artigo também relata redução de 20% na área da placa em um caso específico de matriz 4×8 com oito circuitos e placa de seis camadas, não como ganho universal.

9 Desempenho e integração entre AES e SPI

9.1 Taxa de comunicação e tempo de transferência

No SPI convencional de uma linha por sentido, cada ciclo transporta um bit em MOSI e pode transportar outro em MISO. A 1 MHz, o limite bruto é 1 Mbit/s por sentido, ou 125 kB/s usando kB decimal. A simultaneidade não reduz pela metade o número de ciclos necessário para enviar uma sequência em um único sentido.

Um bloco AES de 128 bits exige 128 ciclos para ser transmitido em uma linha SPI convencional. A 1 MHz, isso corresponde a 128 microssegundos. A expressão usada é tempo = número de bits dividido pela frequência do clock. Esse valor desconsidera comando, endereço, pausas, ativação de CS e processamento interno. Se entrada e saída forem realizadas em transações separadas, serão pelo menos 256 ciclos para os dois blocos.

A frequência nominal do barramento não é a taxa útil da aplicação. Ler estado, enviar comandos e aguardar um resultado consome tempo. Além disso, a taxa de produção do circuito AES pode ser menor ou maior que a taxa de transporte por SPI. Buffers e o protocolo de controle precisam acomodar essa diferença.

9.2 Exemplo de um periférico criptográfico

Considere, como cenário de projeto, um microcontrolador conectado por SPI a um circuito que executa AES. O controlador envia um comando e um bloco de 16 bytes; o periférico monta o estado, executa as rodadas e disponibiliza o bloco resultante. A chave pode ter sido provisionada anteriormente. Essa descrição é uma possibilidade de integração, e não um protocolo definido pela FIPS 197 ou pelo artigo de SPI.

O projeto precisa especificar a ordem dos bytes, a variante AES, o formato dos comandos e uma indicação de ocupado ou pronto. Como o resultado depende de todo o bloco de entrada e do processamento, ele não estará necessariamente disponível enquanto o mesmo bloco ainda está sendo recebido. MISO pode carregar estado, preenchimento ou um resultado anterior, conforme o protocolo.

Se SCLK e o clock interno do núcleo AES forem diferentes, a passagem de comandos e dados entre eles requer tratamento de cruzamento de domínios de clock. Uma solução deve garantir que o bloco seja recebido por inteiro e permaneça estável durante sua captura. Essa preocupação pertence à integração digital, além das regras do algoritmo e da interface.

9.3 Limites de segurança e confiabilidade

O SPI básico não oferece cifragem, autenticação ou detecção de erros obrigatória. Um protocolo de dispositivo pode acrescentar CRC, status e outras verificações, mas esses recursos precisam ser confirmados. Tampouco existe um formato universal de comando ou uma frequência máxima única para todos os componentes SPI.

Usar AES em um equipamento não protege automaticamente todos os sinais da placa. Se texto claro ou chaves forem enviados pelo SPI, esses valores estarão presentes nas linhas durante a transferência. A arquitetura deve definir onde os dados precisam permanecer confidenciais, onde as chaves ficam armazenadas e como a integridade das mensagens é verificada.

10 Conclusões

10.1 Síntese dos conteúdos

O AES organiza blocos de 128 bits em uma matriz de bytes e aplica uma sequência definida de substituições, permutações, misturas e combinações com chaves de rodada. As variantes diferem no tamanho da chave, na expansão e no número de rodadas. Entender a organização por colunas e a ausência de MixColumns na rodada final é decisivo para interpretar corretamente seu funcionamento.

O SPI coordena a transferência serial por clock, seleção e linhas de dados. Seu funcionamento depende da concordância sobre CPOL, CPHA, ordem dos bits e formato da transação, além do respeito aos limites elétricos e temporais. A configuração regular facilita o acesso individual; a

cadeia reduz seleções, mas exige suporte do dispositivo e mais ciclos para percorrer seus estágios.

Aspecto	AES	SPI
Papel principal	Transformação criptográfica de dados	Comunicação serial entre circuitos
Elemento de controle	Chave e sequência de rodadas	Clock, CS e formato de transação
Unidade típica	Bloco fixo de 128 bits	Bits e palavras definidos pelo dispositivo
Verificação essencial	Vetores conhecidos e etapas intermediárias	Formas de onda e requisitos da ficha técnica
Limite do recurso isolado	Não define uma proteção completa de mensagens	Não fornece segurança criptográfica

Em conjunto, os dois recursos permitem construir sistemas que processam informação criptográfica e a movimentam entre componentes. O resultado depende da integração: o núcleo AES deve produzir os valores corretos, a interface deve transferi-los sem erro e o protocolo deve preservar os requisitos de segurança da aplicação.

11 AES Top Level

Resumo da arquitetura e do funcionamento

O AES Top Level integra o núcleo de criptografia AES com os blocos necessários para controlar operações, movimentar dados e inicializar o sistema. No projeto apresentado, o SPI configura e controla o acelerador, enquanto a interface de memória recebe os dados e devolve os resultados.

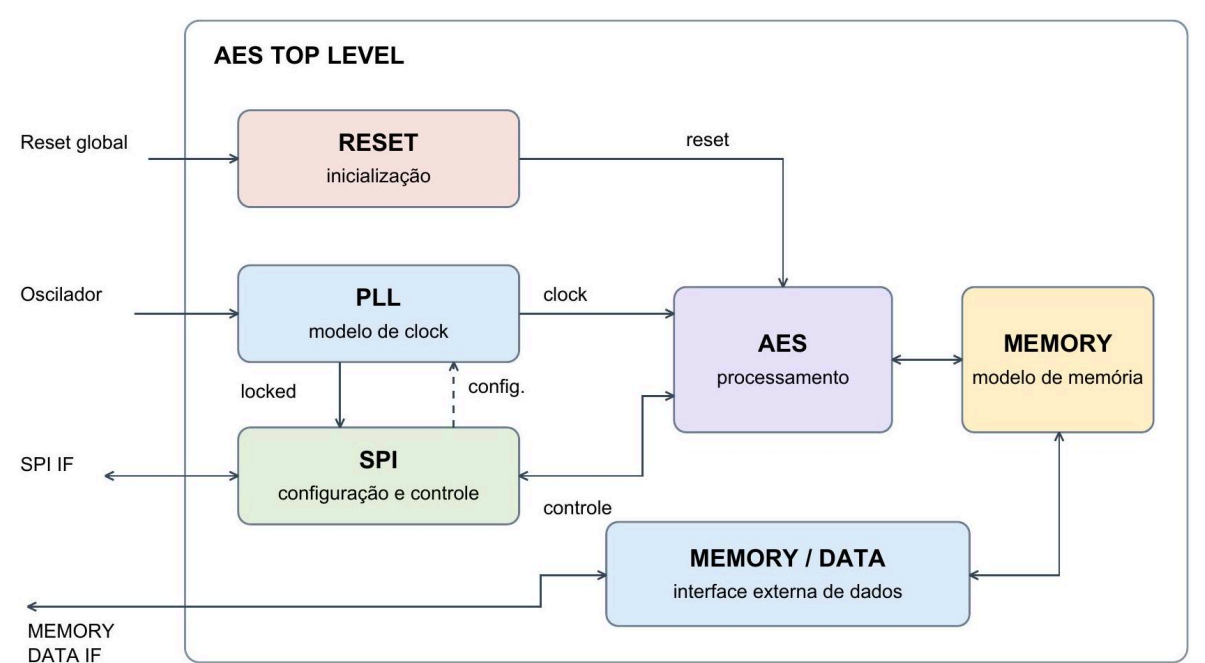


Figura 1. Visão simplificada da arquitetura fornecida. As setas representam conexões funcionais; os sinais internos ainda precisam ser especificados.

11.1 Função de cada bloco

Bloco	Responsabilidade
AES	Executa a transformação criptográfica de blocos de 128 bits usando uma chave secreta.
SPI	Recebe configurações e comandos; pode devolver informações de estado.
MEMORY	Armazena dados utilizados e produzidos pelo processamento.
MEMORY / DATA	Conecta a interface externa de dados à memória.
RESET	Coloca o controle em uma condição inicial conhecida.
PLL	Representa a geração de clock; locked indica estabilidade. Aparece como modelo no diagrama.

11.2 Como o sistema funciona

Sequência típica de uma operação

A sequência abaixo ilustra a integração dos blocos. Os comandos, endereços e sinais exatos dependem da especificação do projeto.

1. Inicializar. O reset estabelece um estado conhecido. O sistema aguarda as condições de clock necessárias antes de liberar o processamento.
2. Configurar. O sistema externo envia os parâmetros de operação pelo SPI. A chave deve estar disponível por um mecanismo de carregamento definido no projeto.
3. Carregar os dados. A interface MEMORY / DATA recebe os bytes de entrada e os disponibiliza na memória.
4. Processar. Após um comando de início, o AES lê o bloco, executa as rodadas da cifra e produz o resultado. O controle deve impedir alterações indevidas durante a operação.
5. Recuperar o resultado. A saída é armazenada e disponibilizada pela interface de dados. Uma indicação de conclusão pode ser consultada pelo SPI, conforme o protocolo adotado.

11.3 O que acontece dentro do AES

O AES trabalha sempre com blocos de 128 bits, ou 16 bytes. As variantes AES-128, AES-192 e AES-256 usam chaves de tamanhos diferentes e executam 10, 12 e 14 rodadas, respectivamente. O núcleo combina substituição de bytes, deslocamento de linhas, mistura de colunas e XOR com chaves de rodada. O top level coordena esse processamento, sem modificar o algoritmo.

11.4 Baixo consumo

Quando não há trabalho, o projeto pode reduzir atividade e interromper o clock do núcleo com clock gating. Desligar sua alimentação, por power gating, exige cuidados adicionais com isolamento e preservação do estado. O UPF descreve a intenção de potência. Essas técnicas são possibilidades de implementação; o diagrama não comprova sua aplicação nem uma economia medida.

11.5 Cuidados principais de integração

É necessário definir a ordem dos bytes, o compartilhamento da memória, a aceitação dos comandos e o momento em que a saída se torna válida. Se SPI e AES usam clocks diferentes, a transferência entre eles precisa de sincronização adequada. Como o SPI configura o PLL, a inicialização deve permitir essa configuração sem depender de um clock ainda indisponível.

Conclusão

O AES realiza a criptografia; o top level organiza o sistema. A integração entre controle SPI, memória, reset e clock permite receber dados, processá-los e entregar resultados. A arquitetura fornecida é um rascunho: mapa de registradores, variante AES, latências e modos de energia ainda precisam ser definidos.